

15 — Case Study: Custo de Memória de Kernels de Normalização

Pré-requisito: todo o resto deste módulo, especialmente arquivos 06, 07, 08 e 14.

Isto ainda não é CUDA. Nenhuma linha de código deste arquivo é kernel — é raciocínio de performance aplicado ao problema exato do seu North Star de pesquisa ([research/north-star.md](#)), usando só as ferramentas que este módulo já construiu: contar FLOPs, contar bytes, contar passadas, calcular AI, decidir save vs recompute. Nenhum número aqui é medido — são contagens simbólicas (em função de n , o tamanho do vetor), exatamente como o `vector_add/matmul` dos arquivos anteriores.

1. Por que este arquivo existe

Todo o módulo até aqui te preparou pra fazer uma pergunta específica sobre qualquer operação: "quantos FLOPs, quantos bytes, quantas passadas, qual AI, memory-bound ou compute-bound?" Este arquivo aplica exatamente essas perguntas ao problema real do seu lab — normalização — **antes** de você escrever qualquer CUDA, pra que quando a Phase 3 chegar, você já saiba o que está tentando conseguir, não só como escrever sintaxe de kernel.

2. RMSNorm: a fórmula

```
rms = sqrt( mean(x²) + eps )
y   = (x / rms) * weight
```

Onde x é um vetor de n elementos (uma linha de ativação, tipicamente a dimensão "hidden" de um Transformer), `weight` é um vetor de n parâmetros aprendidos, y é a saída, mesmo tamanho de x .

3. Estratégia A: duas passadas (a forma "óbvia" de implementar)

```
PASSADA 1 – calcular a estatística:
  ler x inteiro (n elementos)
  calcular soma de x² (n multiplicações + ~n somas)
  calcular mean, depois sqrt -> rms (custo O(1), irrelevante em n)

PASSADA 2 – reler x e normalizar:
  reler x inteiro (n elementos, DE NOVO)
  ler weight (n elementos)
  calcular y = (x/rms) * weight (n divisões + n multiplicações)
  escrever y (n elementos)
```

Contagem de bytes (float32, 4 bytes cada):

Leituras: x (passada 1) + x (passada 2, DE NOVO) + weight = $3n$ elementos = $12n$ bytes
 Escritas: $y = n$ elementos = $4n$ bytes
 Total: $16n$ bytes

Contagem de FLOPs: $\sim n$ (elevar ao quadrado) + $\sim n$ (somar) + $\sim n$ (dividir) + $\sim n$ (multiplicar por weight) $\approx 4n$ FLOPs.

$$\text{AI_A} = 4n / 16n = 0.25 \text{ FLOPs/byte}$$

Repare: esse número é **idêntico** à AI do matmul ingênuo que você já calculou no arquivo 07 — não é coincidência de contexto, é o mesmo tipo de operação estruturalmente (poucos FLOPs por byte movido). Isso confirma o que o [research/north-star.md](#) do seu próprio lab já assume: **normalização é memory-bound por natureza matemática**, não por má implementação.

4. Estratégia B: uma passada (fusão / redução de materialização)

A pergunta que motiva esta estratégia: **a segunda leitura de x na Estratégia A é estritamente necessária?** Matematicamente, não — o valor de x não muda entre a passada 1 e a passada 2. Se, durante a execução de um único kernel, x (ou o bloco de x que uma unidade de trabalho está processando) puder ser mantido em memória rápida (registrador ou memória compartilhada, o equivalente GPU de L1 — arquivo 04) entre o cálculo da estatística e a normalização, a segunda leitura de x **da memória lenta** deixa de ser necessária — ela é servida da memória rápida, "de graça" (ou quase).

UMA PASSADA (conceitual — a viabilidade real depende de x caber em memória rápida disponível, uma restrição de hardware que só a Phase 2/3 vai deixar concreta):

```
ler x da memória lenta UMA VEZ (n elementos), mantendo em memória rápida
calcular soma de  $x^2$  a partir da cópia em memória rápida
calcular rms
calcular y a partir da MESMA cópia de x em memória rápida (sem reler da memória
lenta)
ler weight (n elementos)
escrever y (n elementos)
```

Contagem de bytes:

Leituras (da memória lenta): x (uma vez) + weight = $2n$ elementos = $8n$ bytes
 Escritas: $y = n$ elementos = $4n$ bytes
 Total: $12n$ bytes

FLOPs: os mesmos $4n$ de antes — a matemática não mudou, só a forma de buscar o dado.

$$AI_B = 4n / 12n \approx 0.33 \text{ FLOPs/byte}$$

PERGUNTA RÁPIDA 1: **AI_B** é maior que **AI_A** (0.33 vs 0.25). Usando o que você aprendeu no arquivo 07 (seção 6), isso veio de aumentar o numerador (mais FLOPs) ou de diminuir o denominador (menos bytes)?

► Resposta

5. O que isso NÃO prova (leia com atenção)

Esse raciocínio mostra que existe uma **oportunidade teórica** de reduzir tráfego de 16n pra 12n bytes (25% de redução) — não prova que essa oportunidade é **fácil de realizar** em hardware real, nem que já foi realizada. Viabilidade real depende de detalhes que só a Phase 2/3 vão te dar vocabulário pra avaliar: **x** cabe em shared memory/registadores disponíveis pro tamanho de bloco escolhido? O custo de sincronização entre as duas "fases" do kernel (calcular estatística, depois normalizar) supera o ganho de tráfego evitado? Essa é exatamente a pergunta que implementações reais de kernels fundidos de normalização (existentes publicamente, mencionadas no seu [research/north-star.md](#) como prior art a estudar — Apex, Liger Kernel, o layer_norm do flash-attention) resolveram na prática — mas você ainda não implementou nem verificou nada disso, só identificou **onde a oportunidade matemática mora**.

6. LayerNorm: mais uma estatística, mais uma passada potencial

```
mean = mean(x)
var = mean((x - mean)²)
y = (x - mean) / sqrt(var + eps) * weight + bias
```

Diferente do RMSNorm (uma estatística: **mean(x²)**), LayerNorm precisa de **duas** estatísticas: **mean** e **var**. Isso muda a contagem de passadas na implementação mais ingênua:

```
PASSADA 1: ler x, calcular mean(x)
PASSADA 2: reler x, calcular var usando mean já calculado -- mean((x-mean)²)
PASSADA 3: reler x DE NOVO, calcular y = (x-mean)/sqrt(var+eps)*weight + bias
```

Três leituras de x na versão mais ingênua — pior que as duas do RMSNorm, porque a dependência entre **mean** e **var** (você precisa do **mean** pronto antes de poder calcular **var**) parece forçar uma passada extra.

PERGUNTA RÁPIDA 2: existe uma identidade matemática — **var(x) = mean(x²) - mean(x)²** — que permite calcular **mean** e algo equivalente a **var na mesma passada**, acumulando **sum(x)** e **sum(x²)** simultaneamente. Quantas leituras de **x** isso reduziria o total para, comparado às 3 da versão ingênua?

► Resposta

7. As perguntas de raciocínio que você deve saber fazer, pra qualquer normalização nova

Generalizando o que os dois casos acima mostraram, quando você encontrar uma variante nova de normalização (RMSNorm, LayerNorm, GroupNorm, ou o que for), as perguntas certas são sempre as mesmas:

1. **Quantas reduções (estatísticas agregadas) essa normalização precisa calcular?** Cada uma é, em potencial, uma passada sobre o dado.
2. **Alguma dessas reduções pode ser combinada numa passada só** (como `mean` + `var` via soma e soma dos quadrados)?
3. **O dado de entrada precisa ser relido**, ou pode ser mantido em memória rápida entre o cálculo da estatística e o uso dela?
4. **Quais intermediários (estatísticas escalares, tipicamente pequenas) vale a pena salvar** — porque são baratos de guardar e caros de recalculer — versus **quais dados grandes** (como `x` inteiro) seria melhor reler/recomputar em vez de manter guardado (trade-off do arquivo 06, *save vs recompute*, agora aplicado a um caso concreto)?

8. Prévia: por que o backward pode ser mais difícil

Tudo acima foi só o **forward** (calcular `y` a partir de `x`). Treinar uma rede exige também o **backward** — calcular os gradientes. Sem entrar em derivação nenhuma aqui (isso é o assunto dedicado da Phase 7 do seu [ROADMAP.md](#), "Backward Kernels", a "habilidade central do lab"), vale plantar, conceitualmente, por que essa parte tende a ser mais complexa:

- **Gradiente em relação à entrada** (dL/dx) tipicamente precisa combinar o gradiente que chega de cima (dL/dy) com informação da passada forward — seja `x` original (relido, ou mantido salvo), seja as estatísticas (`rms`, ou `mean/var`) já calculadas antes.
- **Gradiente em relação aos parâmetros** ($dL/dweight$, e $dL/dbias$ no caso de LayerNorm) exige uma **redução ao longo do batch inteiro** — somar a contribuição de gradiente de cada exemplo do lote pra cada peso — um eixo de redução **diferente** do eixo usado no forward (que reduzia ao longo da dimensão hidden, dentro de uma única linha).
- A decisão de **salvar vs recomputar** intermediários (seção 7, pergunta 4) reaparece aqui com força total: estatísticas pequenas (`rms`, `mean`, `var`) quase sempre valem a pena salvar da passada forward; `x` inteiro, sendo grande, é uma decisão menos óbvia — pode valer mais recomputar certas coisas no backward do que pagar o custo de memória de manter tudo salvo (esse é, literalmente, o assunto central do seu [research/north-star.md](#)).

Você vai estudar isso a fundo depois — a única coisa que este arquivo pede é que você reconheça, desde já, que o backward não é "o forward ao contrário e pronto": ele introduz um eixo de redução novo, mais decisões de *save-vs-recompute*, e mais dependência entre passos.

9. Por que isso importa para CUDA (e é, literalmente, o motivo deste lab existir)

Este arquivo inteiro é uma prévia, em raciocínio puro, do que a Phase 6 (Deep Learning Kernel Fundamentals), Phase 7 (Backward Kernels), Phase 8 (LayerNorm e RMSNorm — "research milestone 1 e 2") e Phase 11 (Fused Normalization Kernels — "research milestone 3 e 4") do seu [ROADMAP.md](#) vão desenvolver de verdade, com CUDA/Triton, medição real, e profiler. A diferença entre a Estratégia A e B deste arquivo — reler `x` vs mantê-lo em memória rápida — é, em essência, a pergunta de pesquisa por trás do título do seu North Star: "*Memory-Efficient Fused Normalization Kernels for Transformer Training*". Você acabou de fazer, em miniatura, com papel e caneta, o mesmo tipo de raciocínio que vai sustentar meses de trabalho de pesquisa mais à frente.

10. Erros comuns

- Achar que "estratégia B tem AI maior, logo já é melhor implementada e resolvida" — é uma **oportunidade teórica**, não uma implementação verificada (seção 5).
- Ignorar a ressalva de estabilidade numérica do truque de uma passada pra LayerNorm ($\text{mean}(x^2) - \text{mean}(x)^2$) — é uma técnica real, mas com um trade-off de precisão que merece cuidado, não aplicação cega.
- Pular direto pra tentar otimizar backward sem entender forward primeiro — a ordem do seu próprio `ROADMAP.md` (forward antes de backward, em quase toda fase) não é arbitrária.

PREDICT BEFORE RUNNING (raciocínio, sem código)

Considere uma normalização hipotética que precisa de **três** estatísticas independentes (não combináveis numa fórmula tipo $\text{mean}(x^2) - \text{mean}(x)^2$), cada uma exigindo sua própria passada sobre x , seguida de uma passada final de normalização.

1. Quantas leituras de x a versão mais ingênua exigiria?
2. Quantos bytes totais (leituras + escrita de y , ignorando `weight/bias` por simplicidade), em função de n ?
3. Se, por alguma restrição de hardware, só fosse possível manter x em memória rápida durante **duas** das quatro passadas (não as quatro), quantas leituras da memória lenta você ainda economizaria, comparado à versão totalmente ingênua?

► Explicação